

Action Recognition on HMDB51

- Shen Hao Stefano Lin
[MAT. 881525]
s.lin2@campus.unimib.it
- Hao Li
[MAT. 947013]
l.hao1@campus.unimib.it

Problem Introduction

- The Dataset, HMDB51
- Classification Problem
- Core Challenges

spatial

temporal

the data, size and quality

Dataset Overview

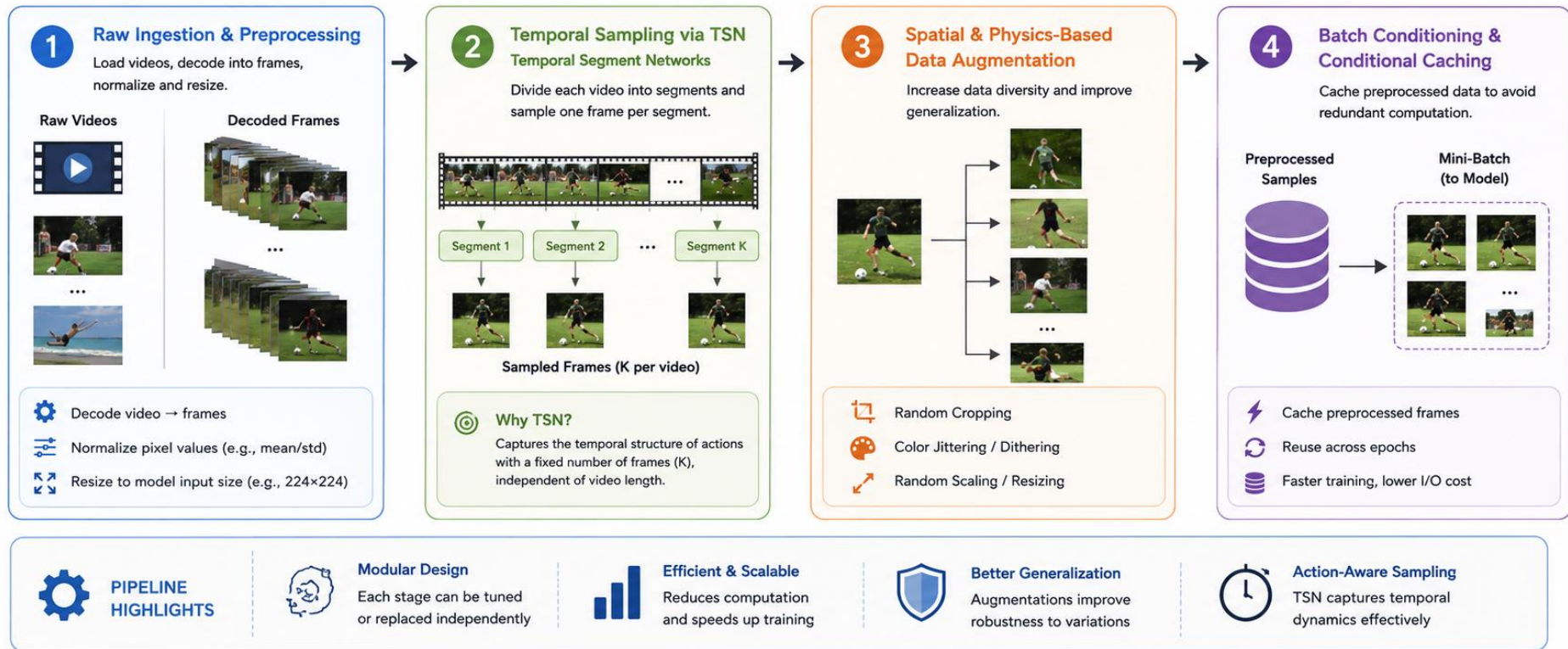
- **Scale: 51 classes, ~6800 clips**
- **Source: Real-world, unconstrained videos**
(Movies, YouTube)
- **Split already: training, validation, test**

Data Pipeline

- **Raw Ingestion & Preprocessing**
- **Temporal Sampling (TSN)**
- **Spatial & Physics Augmentation**
- **Batching & Caching**

Structured Data Pipeline for HMDB51

From raw videos to model-ready batches

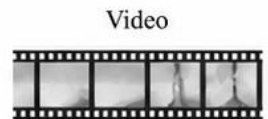


* Illustration AI generated

Solution: Overall Architecture

- **Architecture: CNN+RNN**
- **Other Possible Solutions: 3D-CNN, Video Transformer...**
- **Expected Accuracy: 55%~70%**

Data Loader (Video Sampling)



Video

Frame 1



Frame 2



⋮

Frame T-1

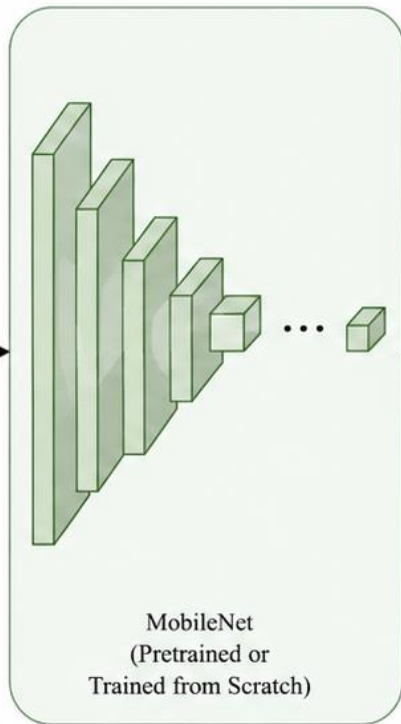


Frame T

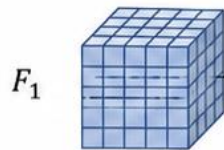


Sampled Frame Sequence
(Length = T)

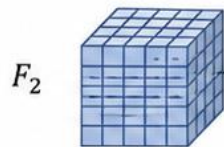
CNN (MobileNet)



Feature Maps

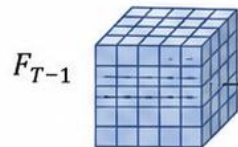


F_1



F_2

⋮



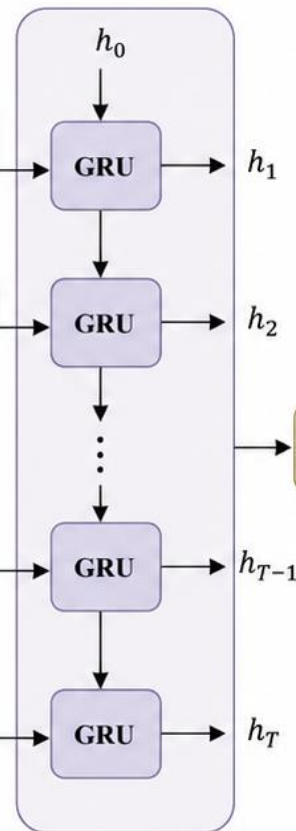
F_{T-1}



F_T

Feature Map Sequence
(Length = T)

RNN (GRU)



Hidden State Sequence

Classification Output



$\begin{bmatrix} p_1 \\ p_2 \\ \dots \\ p_C \end{bmatrix}$

Predicted Class

Class Probabilities
(C classes)

$\text{argmax}(p_i)$
(Predicted Class)

* Illustration AI generated

```
# Two-Stream Architecture: MobileNetV2 (Spatial) + GRU (Temporal)
```

```
inputs = tf.keras.Input(shape=(FRAMES, IMAGE_SIZE, IMAGE_SIZE, 3))
```

```
# Stage 1: Spatial Feature Extraction |
```

```
backbone = MobileNetV2(input_shape=(IMAGE_SIZE, IMAGE_SIZE, 3),  
                        include_top=False, weights='imagenet', pooling='avg')  
backbone.trainable = False # Frozen in Stage-1
```

```
x = TimeDistributed(backbone)(inputs) # → (batch, frames, 1280)
```

```
x = LayerNormalization()(x)
```

```
# Stage 2: Temporal Modeling
```

```
x = Bidirectional(GRU(128, dropout=0.4))(x) # → (batch, 256)
```

```
# Classifier Head
```

```
x = Dropout(0.4)(x)
```

```
x = Dense(256, activation='relu')(x)
```

```
outputs = Dense(51, activation='softmax')(x) # 51 HMDB classes
```

```
model = tf.keras.Model(inputs, outputs, name='mobilenet_gru')
```


Solution: Two-Stage Fine-Tuning Strategy

- **Stage 1: Freeze the CNN, train the RNN**
Large Learning Rate($1e-3$)
- **Stage 2: Unfreeze the CNN, train both**
Smaller Learning Rate($1e-4$)

Two-Stage Fine-Tuning Strategy for CNN + RNN on HMDB51

From pretrained features to task-specialized representation

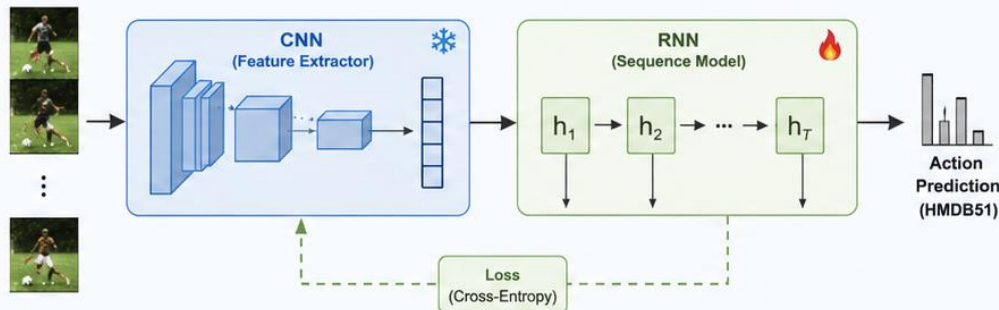
STAGE 1

Train RNN only
(CNN frozen)

❄️ **CNN: FROZEN**
Weights fixed
(ImageNet pretrained)

🔥 **RNN: TRAINED**
Learning rate: $1e-3$

Input: Video (sequence of frames)



💡 INTUITION

The CNN already contains useful feature representations from ImageNet pretraining. Training it too early, before the RNN has learned a good representation, can destabilize those pretrained weights.

➔ Let the RNN learn to interpret the CNN's output first.

➡ RNN converged / stabilized

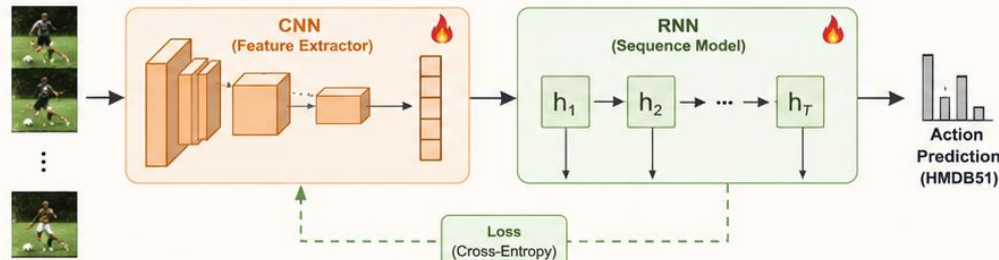
STAGE 2

Fine-tune CNN + RNN
jointly

🔥 **CNN: TRAINED**
Fine-tuned
(small learning rate)

🔥 **RNN: TRAINED**
Learning rate: $1e-5 \sim 1e-6$

Input: Video (sequence of frames)



🎯 GOAL

Adapt the CNN features to the specific visual characteristics of HMDB51 actions while preserving the useful structure learned from ImageNet.

➔ Joint fine-tuning with a small learning rate refines the whole model for the target task.



WHY TWO-STAGE?

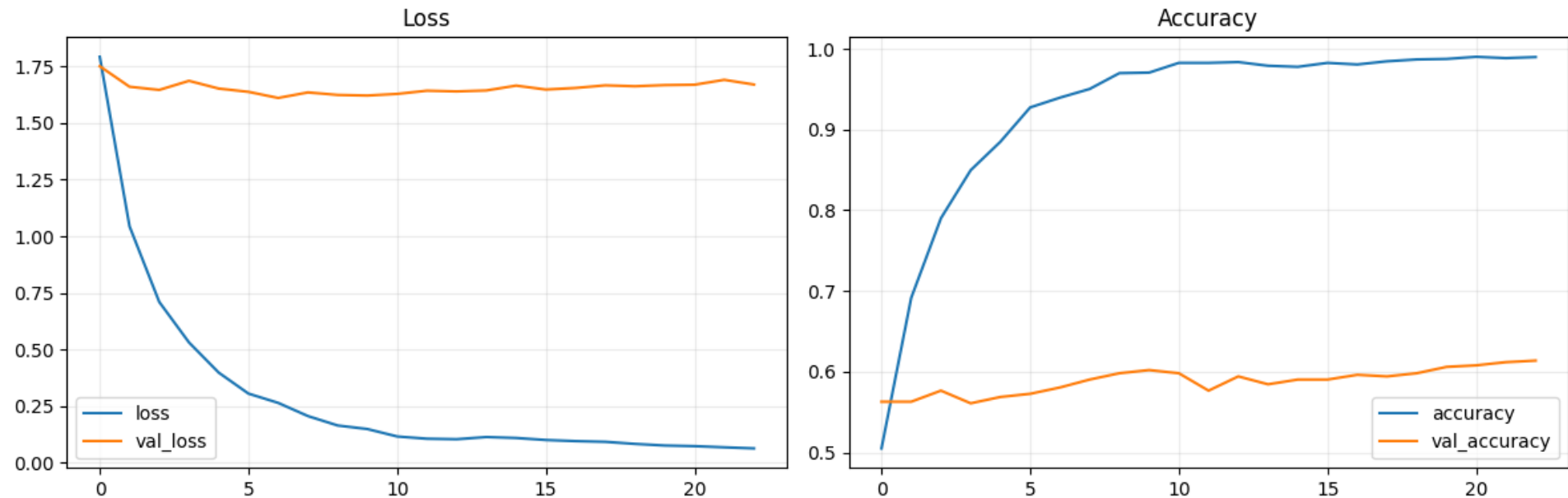
✓ Preserves pretrained knowledge
Avoids destroying useful features early on

✓ More stable training
RNN learns first on a stable feature space

✓ Better performance
Empirically outperforms end-to-end training from scratch

* Illustration AI generated

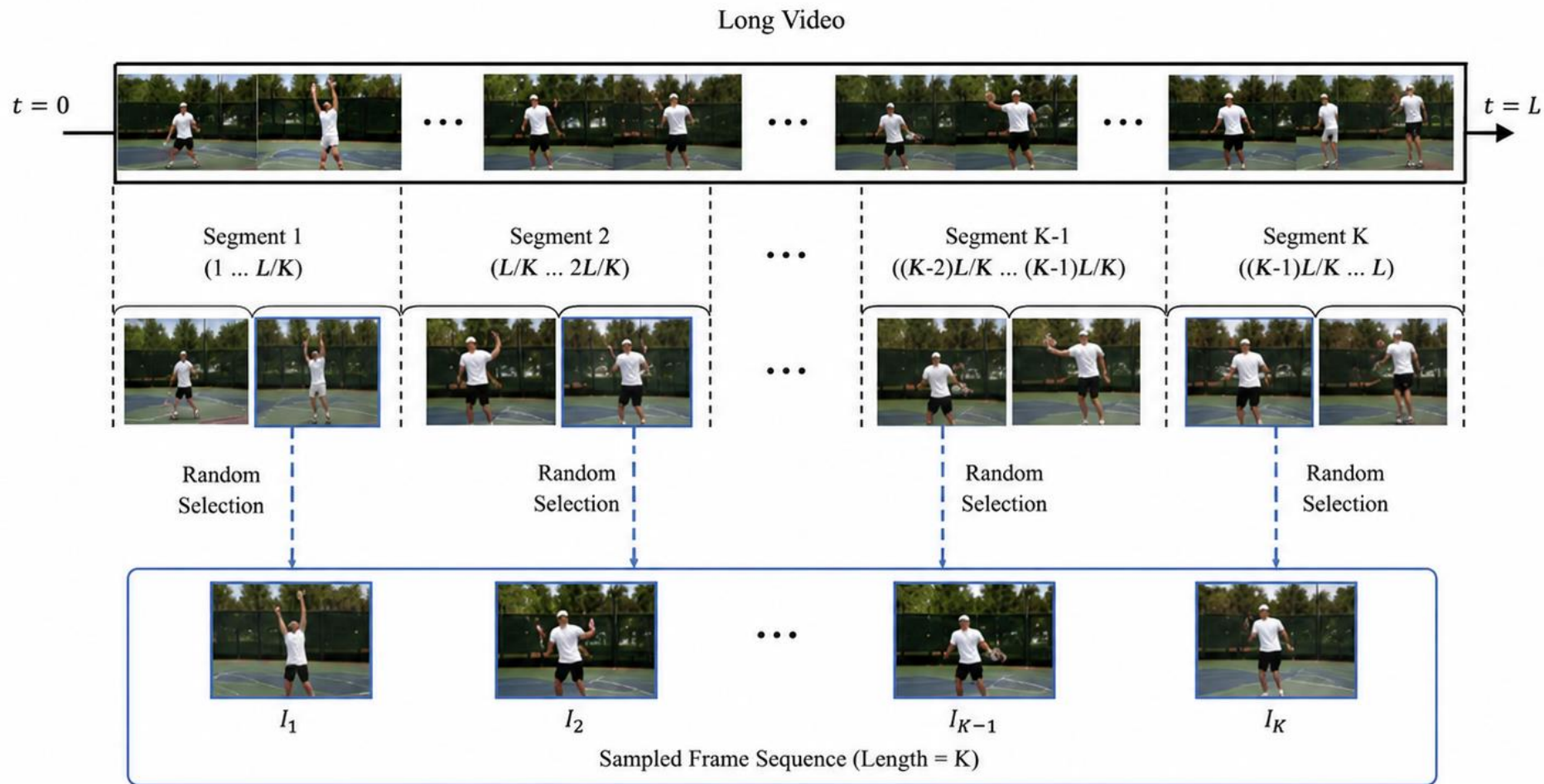
Problem: Overfitting



- Training Accuracy: 99%
- Validation Accuracy: 62% (top 5 Acc around 80%)

Solution: Data Augmentation

- Spatial: cropping, dithering, scaling, *no flipping*
- Temporal: TSN Random Sampling
- Frame Differencing



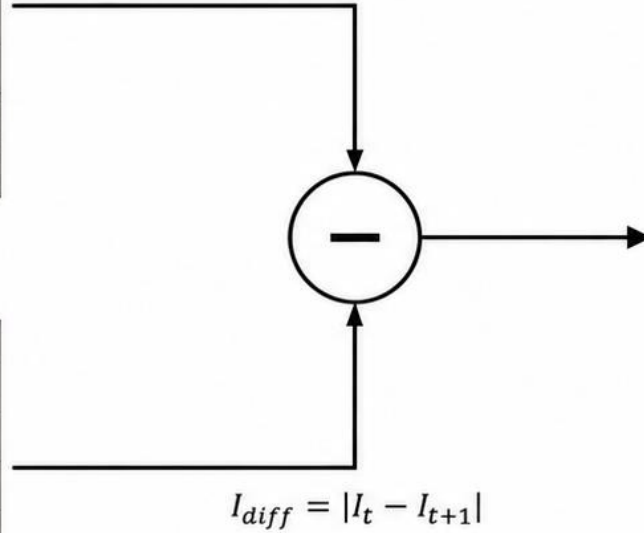
* Illustration AI generated

Frame Differencing for Motion Extraction

Frame I_t (RGB)



Frame I_{t+1} (RGB)



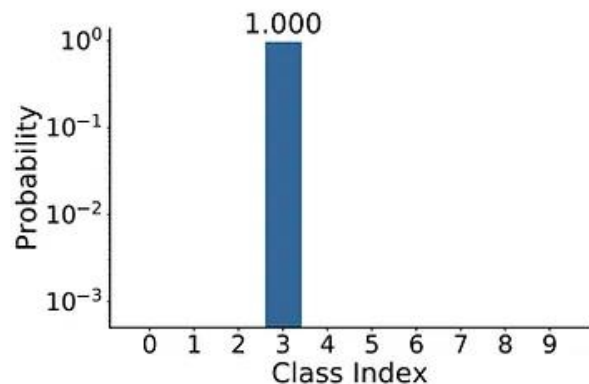
Motion Map (Frame Difference)



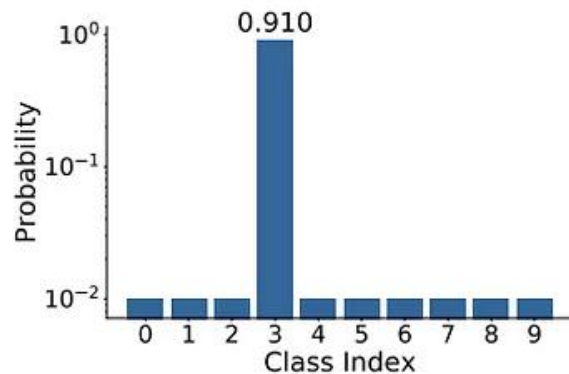
** Illustration AI generated*

Solution: Revise the Model

- **Dropout**
- **Label Smoothing**
- **Feature Bottleneck Layer**
- **Local Unfreezing**



(a) Hard Label



(b) LS

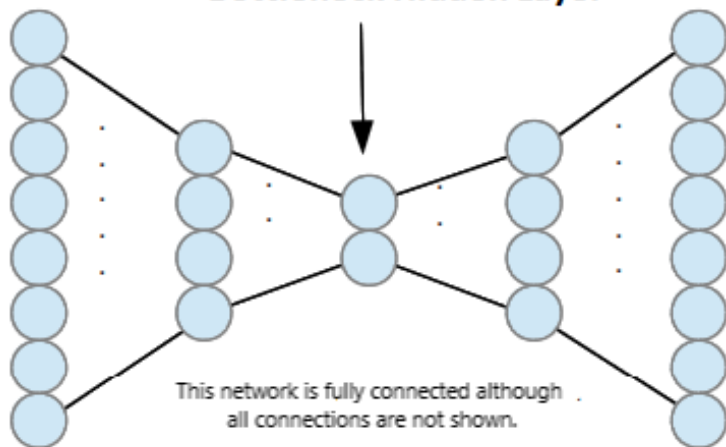
- Label Smoothing

$$p_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

if $p = 1 = \frac{1}{1+0}$

$$z_j \rightarrow \infty, \forall j \neq i$$

Input layer



Output layer
reconstruct input

- Feature Bottleneck Layer

dimensionality reduction

delete some detailed info

Results & Analysis: Accuracy

- Accuracy

training acc 93%

val acc 67.84%

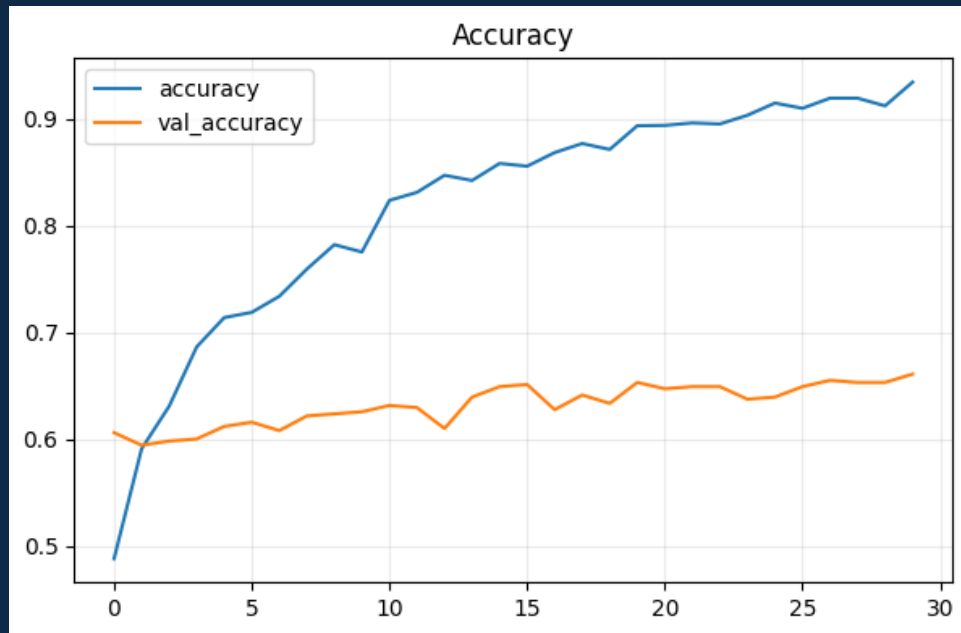
test acc 66.08%

- Top 5 accuracy

training top5 acc 98.63%

val top5 acc 86.27%

test top5 acc 86.47%



Results & Analysis: C.Matrix

- The Diagonal

- The Best

Golf (100%)

Pour (100%)

Pullup (100%)

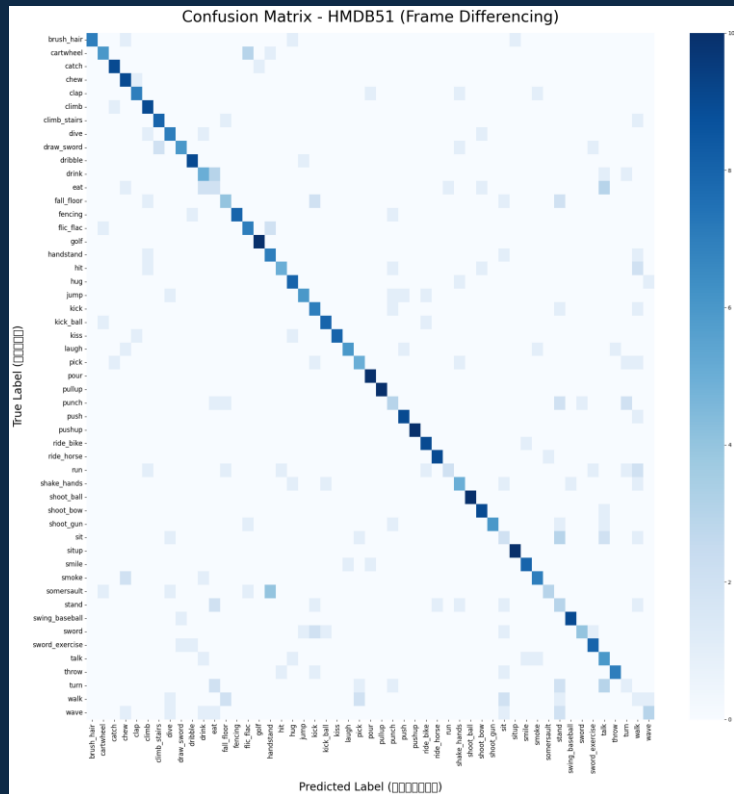
- The Worst

Sit (20%)

Turn (10%)

Walk (10%)

The limitation of Frame differencing ...



Failed Experiments

- L2 Regularization on GRU
- Dynamic Learning Rate (Cosine Decay Restarts)
- Too Large dropout
- Wait to Try: Two-Stream Architecture

Thank You